

Liczby pseudolosowe

1. Liczby pseudolosowe – podstawy

Czasami chcemy wykorzystać liczby pseudolosowe w naszych programach. Idealne rozwiązanie np. do wypełniania tablic losowymi wartościami z podanego zakresu (zamiast wpisywać z klawiatury kilkadziesiąt różnych wartości).

Do tego celu potrzebne nam będą biblioteki:

cstdlib - biblioteka potrzebna do wywołania `srand()` oraz `rand()`:

ctime - biblioteka potrzebna do wywołania funkcji czasu.

Liczby pseudolosowe generowane są na podstawie stanu rejestrów procesora. Powoduje to niestety, przy każdym uruchomieniu programu, generowanie tych samych liczb w tej samej kolejności. Dlatego na początku "mieszymy wartościami rejestrów", ustawiamy tzw. "seeda".

Na samym początku generujemy ziarno, tzw. "seed", dzięki czemu kolejne wartości losowane nie będą miały efektu powtarzalności w kolejnych losowaniach, przy każdym włączeniu programu.

Najlepiej generować na podstawie "czasu", tj. aktualnej liczby sekund jakie upłynęły od roku 1970. Dzięki temu, że to czas jest podstawą "seeda", nie będzie sytuacji, dwa razy wystąpi ten sam seeda.

`srand( time( NULL ) );`

Instrukcja **`rand()`** generuje wartości z przedziału od 0 do 32'767, przez co aby wylosować konkretny przedział musimy wykorzystać mechanizm "reszty z dzielenia".

`int a = rand();`

Zaleca się pierwsze losowanie całkowicie pominąć do obliczeń. Sprawne oko zauważy, że pierwsze losowanie wykazuje tendencję do narastania proporcjonalnie wraz z upływającymi sekundami.

2. Przykłady losowań dla różnych zakresów

Przykładowo, chcemy losować liczby z przedziału od **1** do **10** włącznie. **int b = rand() % 10 + 1;**

W losowanym przedziale mieści się 10 wartości.
rand() % 10 zwraca liczby od 0 do 9 włącznie.
Musimy zatem "przesunąć" o jeden w górę uzyskane wyniki. Dlatego dodajemy 1 do wyniku losowania.

Przykładowo, losujemy liczby z przedziału od **-100** do **+100** włącznie. **int c = rand() % 201 - 100;**

W losowanym przedziale mieści się 201 wartości.
rand() % 201 zwraca liczby od 0 do 200 włącznie.
Musimy zatem "przesunąć" o 100 w dół uzyskane wyniki.
Dlatego odejmujemy 100 od wyniku losowania.

Przykładowo, losujemy liczby rzeczywiste z przedziału od **0.0** do **+1.0** włącznie, z dokładnością do **0.001**
float fp = (rand() % 1001) / 1000.0;

W losowanym przedziale, (dokładność do 0.001, a więc 1000 wartości), łącznie mieści się 1001 wartości.
rand() % 1001 zwraca liczby od 0 do 1000 włącznie.
Musimy zatem podzielić przez 1000.0 uzyskane wyniki (uwaga! **".0"** na końcu, bo dzielenie zmiennoprzecinkowe).

Przykładowo, chcemy (uczciwie) wylosować liczby z przedziału od **1** do **1'000'000** włącznie.
int r = 1000 * (rand() % 1000) + (rand() % 1000) + 1;

Ponieważ **rand()** losuje wartości całkowite zaledwie 15-bitów, musimy „rozciągnąć” taki zakres dla dużych liczb powyżej tego zakresu. Końcowa wartość jest złożeniem (sumą) kilku losowań.

Przykładowy kod. Symulacja 10 kolejnych rzutów kostką o 6 ścianach.

```
#include <iostream>
#include <cstdlib>
#include <ctime>

int main()
{
    srand( time( NULL ) );

    for( unsigned n = 1; n <= 10; n++ )
    {
        int x = rand() % 6 + 1;

        std::cout << "Losowanie nr " << n << " - wypadło oczko: " << x << "\n"; }
    }
```

```
return O;  
}
```